

14: LLM internals, scaling law, inference and serving

Lecturer: Hao Zhang

Scribe: Tongle Shen, Zhiqi Liang, Alex Nguyen, Zihan Zhou, Jaisurya Sundarbabu, Jai Srivastav, Yu Han, Abishek Mahesh

1 LLM Internals

1.1 LLaMA Transformer Decoder Layer Architecture

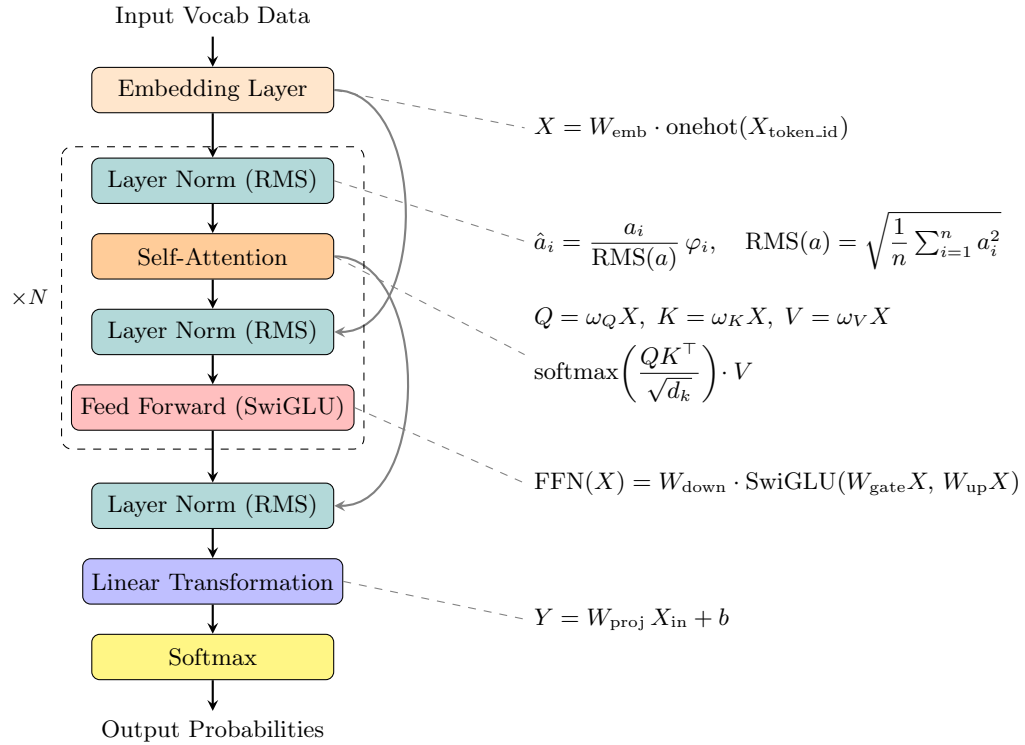


Figure 1: LLaMA Transformer Decoder Layer architecture

A standard LLaMA transformer decoder layer consists of four main components executed in sequence:

Step 1 — Embedding Layer. Raw token IDs are converted to dense vectors via

$$X = W_{\text{emb}} \cdot \text{onehot}(X_{\text{token_id}}),$$

where $W_{\text{emb}} \in \mathbb{R}^{V \times H}$ is the only learnable parameter in this stage (V = vocabulary size, H = hidden dimension). Input vocab data itself carries *no* trainable parameters.

Step 2 — RMS Layer Normalization. Before both the self-attention and feed-forward sub-layers, RMSNorm is applied:

$$\hat{a}_i = \frac{a_i}{\text{RMS}(a)} \varphi_i, \quad \text{RMS}(a) = \sqrt{\frac{1}{n} \sum_{i=1}^n a_i^2},$$

where $\varphi \in \mathbb{R}^H$ is the learnable scale vector. Normalization is critical for stabilizing training and preventing gradient explosion. On multi-GPU systems, RMSNorm is computed locally on each GPU (without an all-gather across the hidden dimension) to avoid expensive communication overhead; the small amount of redundant computation is acceptable in practice.

Step 3 — Multi-Head Self-Attention. The input is projected into queries, keys, and values, followed by scaled dot-product attention:

$$Q = W_Q X, \quad K = W_K X, \quad V = W_V X, \quad \text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

where d_k is the per-head key dimension. Dividing by $\sqrt{d_k}$ prevents the dot products from growing too large, which would saturate the softmax and kill gradients. A final output projection $W_O \in \mathbb{R}^{H \times H}$ maps the concatenated heads back to the hidden space.

Step 4 — Feed-Forward Network with SwiGLU. LLaMA replaces the standard two-layer MLP with a *SwiGLU* variant:

$$\text{FFN}(X) = W_{\text{down}} \cdot \text{SwiGLU}(W_{\text{gate}}X, W_{\text{up}}X),$$

where $W_{\text{gate}}, W_{\text{up}} \in \mathbb{R}^{H \times I}$ and $W_{\text{down}} \in \mathbb{R}^{I \times H}$ (I = intermediate/projection dimension).

After N decoder layers, a final RMSNorm, a linear projection $Y = W_{\text{proj}}X + b$, and a softmax produce the output token probabilities.

1.2 Feed-Forward SwiGLU in Detail

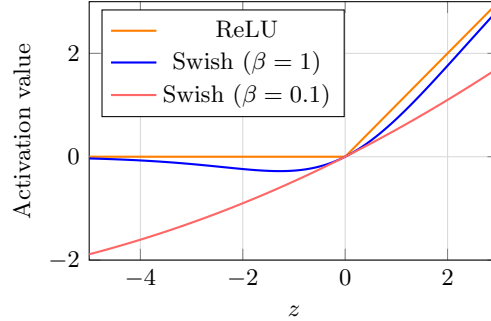


Figure 2: Comparison of ReLU and Swish activation functions

The general SwiGLU formula is:

$$\text{SwiGLU}(x) = \text{Swish}(xW_1 + b_1) \odot (xW_2 + b_2), \quad \text{Swish}(z) = z \cdot \sigma(z),$$

where $\odot$ denotes element-wise multiplication and σ is the sigmoid function.

Why SwiGLU instead of ReLU? ReLU has a sharp discontinuity at $z = 0$: gradients are identically zero for all negative inputs, leading to “dead neuron” pathology. Swish, by contrast, is everywhere differentiable

and has a small but non-zero response for slightly negative values. This smoothness stabilizes gradient flow during training of deep networks and is believed to help LLaMA converge more reliably. SwiGLU also introduces a learned gating mechanism (the element-wise product of two branches) that allows the network to selectively suppress or amplify features, capturing more complex patterns than a plain two-layer MLP.

1.3 Parameter Counting

The parameter count for each component is determined by the shape of its learnable weight matrices. Let V be vocabulary size, H be hidden dimension, N be the number of transformer layers, I be FFN intermediate (projection) dimension:

Table 1: Parameter shapes for each component of the LLaMA decoder.

Component	Parameter(s)	Shape / Count
Embedding Layer	W_{emb}	$V \times H$
RMSNorm ($\times 3$ per layer)	φ (scale vector)	H each
Self-Attention	W_Q, W_K, W_V	$H \times H$ each
	W_O (output projection)	$H \times H$
	Subtotal per layer	$4H^2$
Feed-Forward (SwiGLU)	$W_{\text{gate}}, W_{\text{up}}$	$H \times I$ each
	W_{down}	$I \times H$
	Subtotal per layer	$3HI$
Final Linear (head)	W_{proj}	$V \times H$
Total (approx.)	$2VH + N \cdot (4H^2 + 3HI + 3H) + H$	

Key design consideration: choosing I . The intermediate dimension I significantly impacts both the parameter count and compute cost. In **GPT-3**, $I = 4H$; in **LLaMA 2**, $I \approx 2.73H$. Making I too large inflates both parameters and FLOPs quadratically in H ; making it too small limits the model’s capacity to learn from internet-scale data. Practitioners carefully ablate this ratio before committing to large training runs.

Practical implication. When we say “LLaMA 7B” or “GPT-3 175B”, we are referring to the total parameter count computed from the formula above. This number directly determines the minimum GPU memory required to load and serve the model.

1.4 Estimating Training FLOPs

Knowing the parameter count is not enough; we also need to know the *compute* required to train the model, measured in floating-point operations (FLOPs). This lets us estimate the number of GPUs needed and the expected training duration.

1.4.1 Foundation: Matrix Multiplication FLOPs

Because essentially all computation in a transformer is matrix multiplication, a single formula underpins every estimate in this section. For two matrices $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times h}$:

$$\boxed{\text{FLOPs}(A \times B) = m \times h \times (2n - 1) \approx 2mnh}$$

Each output element $C_{ij} = \sum_{k=1}^n A_{ik}B_{kj}$ requires n multiplications and $n - 1$ additions, giving $2n - 1$ FLOPs. Since n is typically large, we use the approximation $2mnh$. The factor of **2** arises because each *multiply-add* pair counts as two FLOPs.

We adopt the following unified notation for all subsequent derivations:

Symbol	Meaning
b	Batch size (number of sequences)
s	Sequence length (tokens per sequence; assumed uniform)
n	Number of attention heads
d	Hidden-state dimension per head
$h = n \cdot d$	Total hidden-state dimension
i	SwiGLU projection dimension
v	Vocabulary size

The input to the transformer (after embedding) has shape (b, s, h) .

1.4.2 Attention Block FLOPs

Table 2: FLOPs breakdown for one self-attention sub-layer. Red entries are the dominant terms.

Operation	Output Shape	FLOPs	Note
Input X	(b, s, h)	0	—
XW_Q, XW_K, XW_V	(b, s, h)	$3 \times 2bsh^2$	3 matmuls, each $(bs) \times h \times h$
RoPE	(b, n, s, d)	$3bsnd$	position encoding, negligible
$P = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)$	(b, n, s, s)	$2bs^2nd + 3bs^2n$	$QK^\top$: s^2 term; softmax: $3bs^2n$
PV	(b, n, s, d)	$2bs^2nd$	same quadratic scaling as $QK^\top$
AW_O	(b, s, h)	$2bsh^2$	output projection
Residual addition	(b, s, h)	bsh	negligible
Dominant total	—	$\approx 8bsh^2 + 4bs^2nd$	

The s^2 bottleneck. The $QK^\top$ product has shape $(b, n, s, d) \times (b, n, d, s)$. Applying the matmul formula with contraction dimension d and outer dimensions (s, s) gives FLOPs $= 2bns^2d = 2bs^2h$ (since $nd = h$). The same scaling applies to PV . Because s appears *squared*, the attention FLOPs grow **quadratically** with sequence length.

The attention matrix P and memory. After computing $QK^\top$, the intermediate attention matrix P has shape (b, n, s, s) . For $s = 10^6$ this tensor alone would require $b \times n \times 10^{12}$ elements—completely infeasible to materialize in GPU memory. This motivates algorithms like Flash Attention (covered in the next lecture) that avoid ever writing P to HBM.

1.4.3 Feed-Forward (SwiGLU) Block FLOPs

Table 3: FLOPs breakdown for one SwiGLU feed-forward sub-layer.

Operation	Output Shape	FLOPs	Note
Input X (from attention)	(b, s, h)	0	—
$XW_{\text{gate}}, XW_{\text{up}}$	(b, s, i)	$2 \times 2bshi$	two matmuls, each $(bs) \times h \times i$
Swish activation	(b, s, i)	bsi	element-wise, negligible
Element-wise $\odot$	(b, s, i)	bsi	element-wise, negligible
$\cdot W_{\text{down}}$	(b, s, h)	$2bshi$	matmul, $(bs) \times i \times h$
RMSNorm	(b, s, h)	$4bsh + 2bs$	negligible
Dominant total	—	$\approx 6bshi$	

Why the FFN dominates in practice. Substituting $i = 4h$ (GPT-3 setting) into $6bshi$ gives $6bs \cdot h \cdot 4h = 24bsh^2$. In contrast, the attention QKV projections contribute only $6bsh^2$. Hence the FFN is the single largest FLOPs consumer, which is reflected in the measured training cost breakdown.

In LLaMA 2 with $i \approx 2.73h$, the FFN contributes $\approx 16.4bsh^2$, still larger than the QKV projection term.

1.4.4 Total Forward-Pass FLOPs and Concrete Numbers

Summing across N decoder layers and adding the prediction head:

$$\text{FLOPs}_{\text{forward}} \approx N \cdot \underbrace{(8bsh^2 + 4bs^2nd + 3bs^2n)}_{\text{attention block}} + N \cdot \underbrace{6bshi}_{\text{FFN block}} + \underbrace{2bshv}_{\text{prediction head}} \quad (1)$$

LLaMA 2 7B materialization. Using the hyperparameters in Table 4:

Table 4: LLaMA 2 7B hyperparameters used for FLOPs estimation.

Symbol	Description	Value
b	Batch size	1
s	Sequence length	4,096
n	Attention heads	32
d	Per-head dimension	128
$h = nd$	Hidden dimension	4,096
i	FFN projection dim	11,008 ($\approx 2.73h$)
v	Vocabulary size	32,000
N	Number of layers	32

Substituting into Equation 1:

$$\text{FLOPs}_{\text{forward}} \approx \mathbf{63 \text{ TFLOPs}} \quad (b = 1, s = 4096)$$

This is the compute cost of a single forward pass with batch size 1. A 4K context length is considered modest by today’s standards (frontier models support 128K–1M tokens), and even at this scale the compute is already significant.

1.4.5 FLOPs Distribution

Including the backward pass (training FLOPs $\approx 3 \times$ forward FLOPs), the distribution across components is:

Component	Share of Total FLOPs
Embedding layer	1.68%
Normalization	0.007%
Residual connections	0.003%
Attention	41.28%
MLP (FFN)	55.36%
Linear (output head)	1.68%

Conclusion: Attention and MLP together account for $>96\%$ of all training FLOPs. Writing fast, hardware-efficient kernels for these two operations is the primary systems challenge when training or serving LLMs. All other components are negligible and can be optimized later.

1.4.6 Scaling Analysis: Where Is the Bottleneck?

Equation 1 lets us reason about two important scaling regimes:

Scaling model size (increasing h) When h grows, note that both the FFN ($\propto h^2$ via $i \propto h$) and the attention QKV projections ($\propto h^2$) scale *quadratically*. As models grow larger, the FFN term will dominate further, potentially reaching 80–90% of all FLOPs. Optimizing FFN kernels (e.g. fused implementations, quantization) is therefore crucial for large models.

Scaling context length (increasing s) If we hold h fixed and increase s (e.g. to support 1M-token contexts), the FFN FLOPs grow only *linearly* in s , whereas the attention FLOPs grow *quadratically* ($\propto s^2$). At sufficient context length, attention computation overwhelms everything else. This is precisely why techniques such as Flash Attention and linear attention approximations receive so much research attention.

1.5 Memory Estimation Overview

While FLOPs govern *how long* training takes, memory governs *whether* training is feasible on a given cluster. Training memory has four distinct components:

1. **Model Weights.** Determined by the parameter count derived in Section 1.3. Under mixed-precision training (BF16/FP16 forward, FP32 master copy), each parameter occupies $2 + 4 = 6$ bytes.
2. **Intermediate Activation Values.** Every tensor produced during the forward pass that is needed for the backward pass must be held in memory. For each operator, record its output shape and multiply by batch size and number of layers. The attention matrix $P \in \mathbb{R}^{b \times n \times s \times s}$ is a particularly large contributor.
3. **Optimizer States.** The Adam optimizer maintains a first moment (momentum) and a second moment (variance) for every parameter, both stored in FP32. This alone multiplies the weight memory by $2 \times$ on top of the FP32 master copy—yielding roughly $8 \times$ the weight-only footprint.
4. **Weight Gradients + Activation Gradients.** Weight gradients match the shape of the weights and are accumulated during the backward pass. Activation gradients are transient: they can be freed as soon as they have been used to compute upstream gradients.

Gradient checkpointing. One popular technique for reducing activation memory is *gradient checkpointing*: instead of storing all intermediate activations, only a subset of “checkpoint” activations are saved. The rest are recomputed during the backward pass. This trades memory for compute: the backward pass effectively requires one additional forward pass, increasing the training FLOPs multiplier from $3\times$ to $4\times$ the forward FLOPs.

2 Scaling Laws

2.1 Motivation

We know from sections 1.3 and 1.4 that both the amount of compute needed to train a model and its number of parameters are a function of the hidden dimension h and intermediate dimension i . Hence, if we scale up the number of parameters in a model, we will need more compute to train. In addition, the compute is also a function of the batch size b , so the more data we train the model on, the more compute is needed.

But in practice, we have a limited compute budget, so it begs the question: *how large a model should we train, with how much data, to achieve a given performance given our budget?* We need to decide how to allocate limited resources. Scaling laws seek to estimate an answer to this question.

More intuitively, this is a trade-off between training a larger model on less data and training a smaller model on more data. A larger model has more parameters and therefore requires more FLOPs per training step. If the compute budget is fixed, choosing a very large model may force us to reduce the number of training steps, the amount of data, or the effective batch size. In contrast, a smaller model is cheaper per step, so we may be able to train it for longer or on more data.

However, neither choice is always optimal. A large model trained on too little data may be undertrained, while a small model trained on too much data may not have enough capacity to fully use the additional data. Therefore, scaling laws are useful because they quantify this trade-off and help us choose a better balance between model size, dataset size, and compute.

More broadly, scaling laws help answer several practical questions. How large should a model be? How much training data should we use? What performance can we expect under a given compute budget? How should we choose model size and data size before launching an expensive large-scale training run? Instead of training a very large model directly and hoping it works, researchers can train smaller models, fit an empirical relationship, and use that relationship to predict the behavior of larger models.

2.2 The Physics Way

Traditional machine learning and statistics affords an explicit, mathematically derivable answer to the scaling law. For example, suppose we are estimating the mean μ of a normal distribution $N(\mu, \sigma^2)$ by sampling points $x_1, \dots, x_n$, we can mathematically derive the expected error given how many points we sample. Taking logarithms gives a simple linear relationship that the log error decreases as the log number of samples increases. This is a simple scaling law.

$$\hat{\mu} = \frac{\sum_i x_i}{n} \quad E[(\hat{\mu} - \mu)^2] = \frac{\sigma^2}{n} \quad \log(\text{error}) = -\log n + 2 \log \sigma$$

But modern transformer models are far too complex to rigorously derive such clean scaling law mathematically. Instead, LLM scaling laws are usually discovered *empirically* by conducting many experiments. This

is way the lecture describes scaling laws as the "physics way".

In physics, many laws are discovered by running experiments, collecting measurements, and fitting empirical relationships. Similarly, for LLMs, researchers train many small models with different sizes, dataset sizes, and compute budgets, then observe how the loss changes. These empirical laws are not exact mathematical theorems, but they are still extremely useful because they allow researchers to select optimal hyper-parameters based on the scaling law prediction for large models before spending the full training budget.

2.2.1 Transformers vs LSTMs

Scaling laws can also help compare model architectures. For example, one early result shows that transformers scale better than LSTMs. A brute-force way to compare the two would be to train an extremely large LSTM model at the scale of a modern LLM and compare it directly with a transformer, which would be prohibitively expensive. Instead, researchers can train smaller LSTMs and transformers, fit scaling curves, and extrapolate their behavior at larger scales. The scaling law allows us to infer that even if we train a massive size LSTM, it would not outperform a comparable transformer.

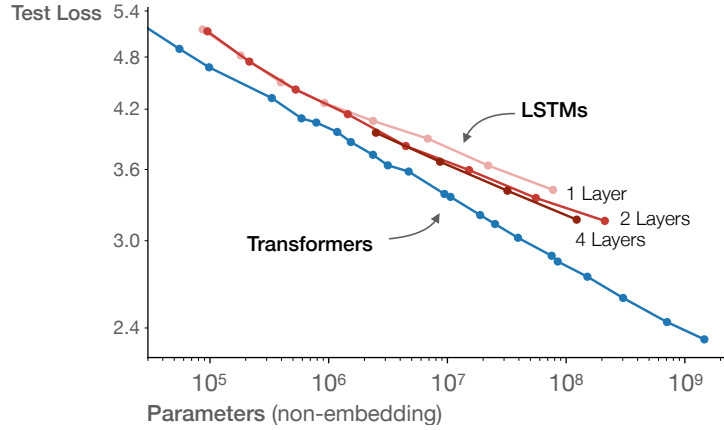


Figure 3: Transformers asymptotically outperform LSTMs due to improved use of long contexts. [1]

2.2.2 Model size data joint scaling

Another form of scaling law seeks to determine the joint relationship between dataset and model size with respect to loss. Joint data-model scaling laws describe how the two relate. One example form is

$$\text{Error} = n^{-\alpha} + m^{-\beta} + c,$$

where one variable represents data size and the other represents model size. Another form from Kaplan-style scaling laws is

$$\text{Error} = [m^{-\alpha} + n^{-1}]^{\beta}.$$

The exact formula may vary across papers, but the core idea is the same: both model size and dataset size matter. If either one is too small, the final performance will be limited.

This law allows us to conclude at what point increasing the amount of data would yield marginal results given a particular model size.

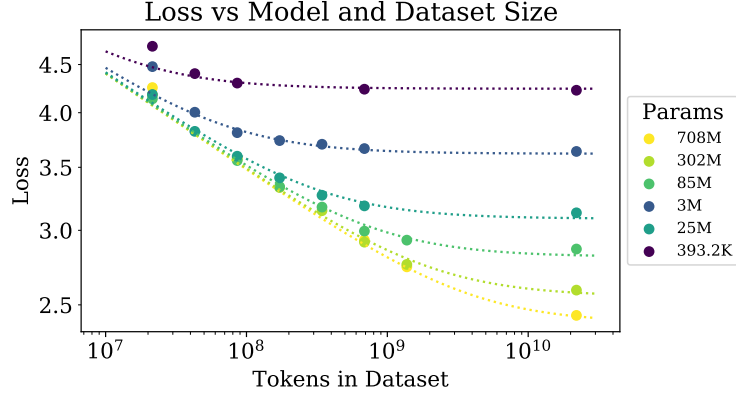


Figure 4: Test loss decreases predictably with the dataset size and model size. [1]

2.2.3 Fixed compute budget

Given a fixed compute budget C , the main question is to choose the optimal model size N and dataset size D that minimize the final loss:

$$N_{\text{opt}}(C), D_{\text{opt}}(C) = \arg \min_{N, D \text{ s.t. } \text{FLOPs}(N, D) = C} L(N, D).$$

Here, N denotes the number of model parameters and D denotes the amount of training data or number of training tokens. The goal is to choose the best combination of N and D under the compute constraint. This formulation captures the practical trade-off discussed above: with the same amount of compute, should we train a larger model for fewer tokens, or a smaller model for more tokens? Scaling laws provide an empirical way to answer this question.

One empirical form for state-of-the-art scaling law shown in the lecture is:

$$L(N, D) = \frac{406.4}{N^{0.34}} + \frac{410.7}{D^{0.29}} + 1.69.$$

This formula shows that increasing either model size or dataset size can reduce loss, but both improvements have diminishing returns. Therefore, simply increasing one factor without considering the other may not be compute-optimal.

The instructor also mentioned that scaling laws are continuously updated. As researchers train new models and collect more experimental results, the empirical law may change. This is another reason why scaling laws are closer to physics-style empirical modeling than to fixed mathematical theorems.

2.3 Key Takeaways

The main takeaway is that scaling laws can be viewed as the “physics of machine learning.” For modern LLMs, it is difficult to derive clean theoretical guarantees from first principles. Instead, researchers train models at different scales, observe empirical patterns, and use these patterns to predict the behavior of larger models.

Scaling laws also changed how people evaluate architectures. It is not enough for an architecture to perform

well at a small scale; we also care about whether it continues to improve as model size, data size, and compute increase. This helps explain why Transformers became dominant in large-scale language modeling.

Finally, scaling laws make ML systems essential. If better performance depends on scaling, then distributed training, hardware efficiency, GPU utilization, data pipelines, and serving infrastructure all become central to machine learning progress. In this sense, scaling laws connect model design with system design.

3 Serving & Inference Optimization

3.1 Introduction

Language model inference is becoming one of the dominant workloads in computing, with a large fraction of global compute likely to be spent on running large language models (LLMs) rather than training them. Even seemingly small efficiency improvements (for ex., a 1% reduction in inference cost or latency) can have huge impact at scale because inference runs continuously in production systems. This motivates careful study of how inference actually works and which parts can be optimized. In this section, the focus is on a few core ideas that underlie many modern “serving and inference optimization” techniques for generative LLMs.

3.2 Recap: Next-Token Prediction

An autoregressive language model is trained to model the joint probability of a sequence by factorizing it into conditional next-token probabilities. For a sequence of tokens $x_1, x_2, \dots, x_T$, the model learns

$$P(x_{1:T}) = P(x_1) P(x_2 | x_1) P(x_3 | x_{1:2}) \cdots P(x_T | x_{1:T-1}).$$

where each factor is “predict the next token given all previous tokens.”

Concretely, during training and inference, the model receives a prefix $(x_1, \dots, x_t)$ and produces a distribution over the next token x_{t+1} . The model is thus best thought of as a conditional next-token predictor: given the context so far (the prompt plus any generated continuation), it outputs logits over the vocabulary, from which a discrete token is sampled or greedily selected.

3.3 Inference process of LLMs

At inference time, the user provides a prompt consisting of multiple tokens, such as “Artificial intelligence is ...”. These tokens are fed through the model, which consists of multiple transformer layers stacked from layer 1 to layer N. After processing the prompt, the model outputs a probability distribution over the next token; sampling from this distribution yields the first generated token (for example, “the”).

This newly generated token is appended to the context and fed back into the model to generate the next token. The process repeats: at each step, the model receives all previous tokens (prompt plus generated tokens so far) and outputs a distribution over the next token, continuing until either a predefined maximum length (e.g., 2048 or 1M tokens, depending on the model and system) is reached or a special end-of-sequence token is generated, signaling termination.

3.4 Prefill vs. Decoding Phases

The inference computation can be conceptually split into two phases, which is crucial for understanding where optimizations apply:

- Prefill phase (0-th iteration)

- The entire prompt (all input tokens) is processed at once through all transformer layers.
- Because the whole prompt is known upfront, the model can parallelize computation over prompt tokens along the sequence dimension, similar to how training uses full sequences.
- The main output of this phase is not only the first next-token distribution but also internal intermediate states (keys and values in attention) that will be reused during decoding.
- Decoding phase (subsequent iterations)
 - After prefill, the model generates tokens one by one.
 - Each decoding step processes only a single new token (the one generated in the previous step), conditioned on the entire history so far.
 - Because future tokens are unknown, generation must proceed sequentially, leading to what is called *autoregressive decoding* - you cannot skip ahead or parallelize across future positions in the same sequence.

From a systems perspective, prefill tends to be more throughput-oriented and can be highly parallelized, while decoding is latency-bound and strictly sequential per sequence, making it a primary target for inference optimization techniques.

3.5 The Key-Value (KV) Cache

Each transformer layer computes scaled dot-product attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V, \quad Q = XW_Q, \quad K = XW_K, \quad V = XW_V.$$

During decode, the model has one new input token, but its query must attend over the K, V of *all previous tokens*. Naively recomputing K, V for the whole context every step is hugely wasteful since the past K, V never change. The **KV cache** fixes this: store per-layer K and V for every past token; each new step only computes Q, K, V for the new token, appends its K, V to the cache, and attends over the cached tensors.

For L layers, n heads of dimension d , batch b , and current length s , total KV memory is

$$\text{KV memory} \propto 2 \cdot L \cdot b \cdot n \cdot s \cdot d \cdot (\text{bytes/element}),$$

the factor 2 being for K and V . The cache grows *linearly* with sequence length.

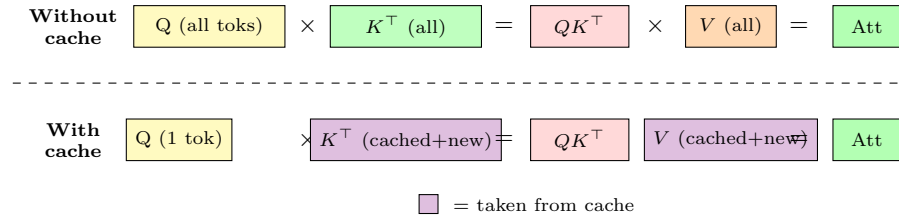


Figure 5: Decode step with vs. without KV cache. With cache, past K, V are reused; only Q, K, V for the new token are computed.

Q1: What happens to the KV cache in prefill? Prefill is when the cache is *populated*. We run full attention over the prompt and *write* the resulting K, V tensors into the cache so that the first decode step can read them. Thus prefill *writes* the cache; decode *reads + appends*.

Q2: Do we need to cache Q ? No. A query is used only at the step it is computed: at decode step t , only Q_t participates in attention. Past queries $Q_1, \dots, Q_{t-1}$ are never reused. Past keys/values, in contrast, are needed by every future query. So we cache K and V only.

3.6 Potential Bottlenecks

A transformer block (used identically in training and inference) is: RMSNorm $\rightarrow$ project to Q, K, V via $W_Q, W_K, W_V \rightarrow \text{softmax}(QK^\top/\sqrt{d}) \cdot V \rightarrow \text{output proj } W_{\text{Output}} \rightarrow \text{RMSNorm} \rightarrow \text{MLP}$ (gate + up + SwiGLU + down). Here b is batch size, s the sequence length for the current step, h hidden size, n heads, d head dim.

Compute. In **prefill**, $s = s_{\text{prompt}}$ and matmuls operate on (b, s, h) -tensors — essentially identical to training, well-utilized on GPU, typically *compute-bound*. In **decode**, $s = 1$, so matmuls collapse to matrix-vector products with very low arithmetic intensity; the GPU spends most of its time reading weights and KV cache from HBM and is therefore *memory-bandwidth-bound*.

Memory. Inference adds a new memory consumer that training does not have: the KV cache. It grows linearly with generated tokens and with batch size, eating into the budget for weights, activations, and additional concurrent requests.

What about batch size b ? Because decode is bandwidth-bound, the natural throughput lever is to raise b : many requests share the same weight reads, lowering per-request cost. But larger b enlarges the KV cache proportionally. The fundamental serving question is therefore: *how large can b be before KV cache exhausts GPU memory?* This motivates later optimizations such as paged attention, continuous batching, KV quantization, and multi-query / grouped-query attention.

Table 5: Prefill vs. Decode at a glance.

Property	Prefill (iter 0)	Decode (iter ≥ 1)
Tokens per step	full prompt	1
Sequence dim s	s_{prompt}	1
KV cache action	populate (write)	read past + append new
Compute pattern	matrix-matrix (like training)	matrix-vector
Typical bottleneck	compute-bound	memory-bandwidth-bound
Effect of batch size b	moderate	large (throughput lever)

3.7 Summary

- An LLM is a next-token predictor: $\text{Max Prob}(x_{1:T}) = \prod_{t=1}^T P(x_{t+1} \mid x_{1..t})$; generation is autoregressive until max length or EOS.
- Inference has two phases: **prefill** (whole prompt, populates cache) and **decode** (one token per step, reads cache).
- The **KV cache** stores per-layer K and V for past tokens; we do *not* cache Q because past queries are never reused.
- Prefill $\approx$ training (compute-bound). Decode has $s = 1$ and is bandwidth-bound; KV cache is the new memory pressure point, growing linearly in sequence length and batch size.
- Batch size b is the main decode-throughput lever but is capped by KV-cache memory — motivating dedicated serving systems and KV-cache optimizations.

References

- [1] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *CoRR*, abs/2001.08361, 2020.